

# BiblioSys — Documentação do Projeto

**Instituição:** Pontifícia Universidade Católica de Minas Gerais  
**Disciplina:** Algoritmos e Estruturas de Dados III  
**Professor:** Walisson Ferreira de Carvalho  
**Grupo:** 9  
**Integrantes:** Fernando Mucci · Maria Eduarda P. Brito · Luan Oliveira · Luiz Felipe Volpe · Luisa Campanha  
**Repositório:** [GitHub — TP-AEDS-III-G9](#)  
**Demonstração:** <https://youtu.be/o8BPHHzVJzI>

## 1. Visão Geral do Projeto

O **BiblioSys** é um gerenciador de acervo bibliográfico que realiza todas as operações de CRUD diretamente em arquivos binários, sem o uso de SGBDs. O backend é em Java, exposto via API REST com servidor HTTP nativo, e o frontend é uma interface web em HTML/CSS/JavaScript puro.

### Modelagem de Entidades

| Entidade | Arquivo de Dados | Relacionamentos                               |
|----------|------------------|-----------------------------------------------|
| Livro    | livros.bin       | N:1 com Editora · N:N com Autor · N:N com Tag |
| Autores  | autores.bin      | N:N com Livro (via livros_autores )           |
| Editora  | editoras.bin     | 1:N com Livro                                 |
| Tag      | tags.bin         | N:N com Livro (via tags_livros )              |
| Usuarios | usuarios.bin     | Autenticação do sistema                       |

## 2. Estrutura do Projeto

```
TP-AEDS-III-G9/
├── src/main/java/
│   ├── Main.java
│   └── src/
│       ├── dao/                # Um DAO por entidade – persistência e indexação
│       │   ├── AutoresDAO.java
│       │   ├── DAOFactory.java
│       │   ├── EditoraDAO.java
│       │   ├── LivroAutorDAO.java
│       │   ├── LivroDAO.java
│       │   ├── TagDAO.java
│       │   ├── TagsLivrosDAO.java
│       │   └── UsuarioDAO.java
│       ├── model/              # POJOs com serialização binária
│       │   ├── Autores.java · Editora.java · Livro.java
│       │   ├── LivroAutor.java · TagsLivros.java · Tag.java · Usuarios.java
│       ├── server/
│       │   └── ApiServer.java    # Servidor HTTP + roteamento REST
│       ├── service/
│       │   └── UsuarioService.java # Autenticação e XOR
│       └── util/
│           ├── Arquivo.java      # CRUD genérico com Best-Fit
│           ├── ArvoreBMais.java  # Árvore B+ em disco
│           ├── HashExtensivel.java # Hash Extensível (chave int)
│           ├── HashExtensivelLong.java # Hash Extensível (chave long – 1:N)
│           ├── OrdenacaoExterna.java # Intercalação balanceada
│           ├── SerializadorUtil.java # String, String[], int[], ISBN
│           └── DataUtil.java · Indexador.java · Registro.java
├── web/
│   ├── index.html · app.js · style.css # Interface Web (SPA)
├── data/
│   ├── *.bin                    # Arquivos de dados das entidades
│   └── idx/                     # Índices: .idx (B+), .dir.bin + .bkt.bin (Hash)
└── pom.xml                      # Maven (Java 17 + org.json)
```

## 3. Armazenamento de Registros

### 3.1 Estrutura dos Registros (Tamanho Variável)

Cada registro é serializado com `DataOutputStream` e gravado pela classe genérica `Arquivo<T>` com o seguinte layout em disco:

```
[ lápide : 1 byte (boolean) ] [ tamanho dos dados : 4 bytes (int) ] [ dados serializados... ]
```

Campos primitivos ( `int`, `long`, `double` ) têm tamanho fixo. Campos `String` são serializados por `SerializadorUtil` com delimitador de byte `;`, resultando em registros de **tamanho variável**.

**Campos por entidade:**

| Entidade   | Campos e Tipos                                                                                                                           |
|------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Livro      | id (int) · idEditora (int) · titulo (String) · isbn (char[13], 13 bytes fixos) · anoPublicacao (int) · preco (double) · sinopse (String) |
| Autores    | id (int) · nome (String) · dataNascimento (long, timestamp Unix ms) · biografia (String)                                                 |
| Editora    | id (int) · nome (String) · demais campos texto (String)                                                                                  |
| Tag        | id (int) · nome (String)                                                                                                                 |
| Usuarios   | id (int) · nome (String) · email (String) · senhaXor (String, Base64) · redesSociais (String[], multivalorado)                           |
| LivroAutor | id (int) · idLivro (int) · idAutor (int) — tamanho fixo total                                                                            |
| TagsLivros | id (int) · idTag (int) · idLivro (int) — tamanho fixo total                                                                              |

### 3.2 Campo Multivalorado — redesSociais em Usuarios

O campo `redesSociais` é um `String[]` serializado por `SerializadorUtil.writeStringArray()`: grava a quantidade de elementos ( `int` ) seguida de cada string com delimitador `;`. A leitura reconstrói o array com `readStringArray()`. Compatibilidade retroativa é garantida: se o campo não existia em registros antigos ( `dis.available() == 0` ), um array vazio é atribuído.

### 3.3 Cabeçalho dos Arquivos .bin

Cabeçalho fixo de **16 bytes** ( `TAM_CABECALHO = 16` ):

| Offset | Tamanho | Conteúdo                                                                 |
|--------|---------|--------------------------------------------------------------------------|
| 0      | 4 bytes | Último ID gerado (autoincremento)                                        |
| 4      | 4 bytes | Total de registros ativos                                                |
| 8      | 8 bytes | Ponteiro da lista de excluídos (long, sentinela <code>NULO = -1</code> ) |

### 3.4 Exclusão Lógica com Reaproveitamento Best-Fit

A exclusão é **lógica**: a lápide é marcada como `false` e o endereço do bloco é inserido em uma **lista encadeada de blocos livres** mantida no cabeçalho. Em novas inserções, `Arquivo<T>` percorre essa lista buscando o **menor bloco que comporte o novo registro** (política *Best-Fit*). O tamanho físico original do bloco é preservado para evitar fragmentação. Se nenhum bloco adequado existe, o registro é anexado ao final.

## 4. Indexação

### 4.1 Hash Extensível por ID (chave int ) — HashExtensivel.java

**Arquivos:** `<entidade>_id.dir.bin` + `<entidade>_id.bkt.bin`

- Bucket com até **10 entradas** ( `TAM_BUCKET = 10` ); cada entrada: ( `chave: int`, `endereço: long` ) = 12 bytes ( `TAM_ENTRADA = Integer.BYTES + Long.BYTES` ).
- Suporta **Profundidade Global e Local**; crescimento por **Split** e **duplicação de diretório**.
- Operações: `inserir`, `buscar`, `remover`, `atualizar`, `getTotalChaves`.
- Complexidade: **O(1) amortizado**.

### 4.2 Hash Extensível para Relacionamento 1:N (chave long ) — HashExtensivelLong.java

**Arquivos:** `livros_por_editora.dir.bin` + `livros_por_editora.bkt.bin`

Variante com entrada de **16 bytes** ( `long chave + long endereço` ). Implementa `listarPorFaixa(chaveMin, chaveMax)` que percorre todos os buckets retornando pares cuja chave está no intervalo — habilitando o range scan por `idEditora` .

### 4.3 Árvore B+ — ArvoreBMais.java

Arquivo: `<entidade>_id.idx`

- **Ordem 4** — até 4 chaves por nó.
- **Nós folha encadeados** para travessias sequenciais crescente e decrescente.
- Tamanho fixo por nó: `TAM_NO = 1 + 4 + ORDEM × (4 + 8) + 8` bytes.
- Cabeçalho: endereço da raiz (offset 0) + total de chaves (offset 8).
- Operações: `inserir` , `buscar` , `remover` , `listarOrdenado()` , `listarOrdenadoDecrescente()` .

## 5. Relacionamento 1:N — Editora → Livros

O `LivroDAO` mantém um índice `hashPorEditora` ( `HashExtensivelLong` ) com chave composta de 64 bits:

```
// 32 bits superiores = idEditora, 32 bits inferiores = idLivro
long chave = ((long) idEditora) << 32 | (idLivro & 0xFFFFFFFFL);
```

Para recuperar todos os livros de uma editora (range scan):

```
long chaveMin = ((long) idEditora) << 32;
long chaveMax = ((long) (idEditora + 1)) << 32;
long[][] pares = hashPorEditora.listarPorFaixa(chaveMin, chaveMax);
```

O agrupamento do `idEditora` nos bits superiores concentra todos os seus livros em faixas contíguas do espaço de chaves.

**Integridade Referencial:** O `EditoraDAO` impede a exclusão de uma editora com livros vinculados, lançando exceção. Ao alterar o `idEditora` de um livro, o `LivroDAO` remove a chave antiga e insere a nova.

## 6. Relacionamentos N:N

### 6.1 Livro ↔ Autor ( livros\_autores.bin )

Tabela `LivroAutor` com campos `(id, idLivro, idAutor)` — todos `int` , tamanho fixo.

Índices: `livros_autores_id` (Hash `int` pelo ID) · `livros_autores_por_livro.idx` (B+ com `idLivro << 16 | idAutor` ) · `livros_autores_por_autor.idx` (B+ com `idAutor << 16 | idLivro` ).

### 6.2 Livro ↔ Tag ( tags\_livros.bin )

Tabela `TagsLivros` com campos `(id, idTag, idLivro)` — todos `int` , tamanho fixo.

Índices: `tags_livros_id` (Hash `int` pelo ID) · `tags_livros_por_livro.idx` (B+ com `idLivro << 16 | idTag` ) · `tags_livros_por_tag.idx` (B+ com `idTag << 16 | idLivro` ).

## 7. Ordenação Externa — OrdenacaoExterna<T>.java

Implementa intercalação balanceada para ordenar por campos que não são chave primária (ex.: título alfabético):

1. **Fase de Distribuição:** Registros ativos lidos em blocos de até `RAM_BLOCOS = 10` registros, ordenados em memória pelo `Comparator<T>` e gravados como *runs* temporários em `data/temp/` .
2. **Fase de Intercalação:** *Runs* fundidos progressivamente (merge externo) até restar um único arquivo ordenado.

O `LivroDAO` expõe três métodos de listagem ordenada:

- `listarOrdenadoPorId()` — travessia crescente das folhas da Árvore B+
- `listarOrdenadoDecrescentePorId()` — travessia decrescente das folhas encadeadas
- `listarOrdenadoPorTitulo()` — usa `OrdenacaoExterna` com `Comparator` por título

## 8. Autenticação e Criptografia XOR — UsuarioService.java

O `UsuarioService` centraliza registro, login e alteração de senha com criptografia implementada manualmente:

```
// Chave derivada: email do usuário + salt estático "AEDs3-G9-2025"
private static String gerarChave(String email) {
    return email + "AEDs3-G9-2025";
}

// XOR byte a byte; resultado em Base64 para armazenamento UTF-8 seguro
public static String xorCriptografar(String texto, String chave) {
    byte[] resultado = new byte[textoBytes.length];
    for (int i = 0; i < textoBytes.length; i++) {
        resultado[i] = (byte) (textoBytes[i] ^ chaveBytes[i % chaveBytes.length]);
    }
    return Base64.getEncoder().encodeToString(resultado);
}
```

O campo `senhaXor` em `Usuarios` armazena o resultado em Base64. A descriptografia é simétrica (XOR aplicado duas vezes recupera o texto original). A chave varia por usuário (derivada do e-mail), impedindo que senhas iguais gerem hashes idênticos.

## 9. Validações de Unicidade

| Campo              | Entidade              | Mecanismo                                                                                                  |
|--------------------|-----------------------|------------------------------------------------------------------------------------------------------------|
| <code>email</code> | <code>Usuarios</code> | <code>UsuarioDAO.buscarPorEmail()</code> — verificado antes de <code>incluir</code> e <code>alterar</code> |
| <code>isbn</code>  | <code>Livro</code>    | <code>LivroDAO</code> — verificado na inclusão e na alteração, impedindo duplicatas no acervo              |

## 10. Formulário de Projeto

### a) Qual a estrutura usada para representar os registros?

Registros de **tamanho variável**, serializados via `DataOutputStream` pela classe genérica `Arquivo<T>`. Layout em disco: `[lápide: 1B][tamanho: 4B][dados...]`. Campos primitivos (`int`, `long`, `double`) são fixos; campos `String` usam delimitador de byte `;` via `SerializadorUtil`. O campo `isbn` em `Livro` é exceção: `char[13]` gravado como 13 bytes fixos por `writeIsbn/readIsbn`.

### b) Como atributos multivalorados do tipo string foram tratados?

A entidade `Usuarios` possui o campo `redesSociais` do tipo `String[]`, serializado por `SerializadorUtil.writeStringArray()` (grava `int` com a quantidade, depois cada string com delimitador `;`). A leitura usa `readStringArray()`. Compatibilidade retroativa: se o campo não existe em registros antigos (`dis.available() == 0`), atribui-se array vazio sem lançar exceção.

### c) Como foi implementada a exclusão lógica?

Pelo primeiro byte de cada registro — a **lápide** (`boolean`). Valor `true` = ativo, `false` = excluído. Ao excluir, o endereço do bloco é inserido em uma **lista encadeada de blocos livres** no cabeçalho (offset 8). Inserções futuras percorrem essa lista pela política **Best-Fit**, preservando o tamanho físico do bloco ao reutilizá-lo.

### d) Além das PKs, quais outras chaves foram utilizadas nesta etapa?

- **Chave estrangeira simples:** `idEditora` em `Livro`.
- **Chaves de tabelas associativas:** `idLivro` + `idAutor` em `LivroAutor`; `idTag` + `idLivro` em `TagsLivros`.
- **Chaves compostas `long` (64 bits):** `(idEditora << 32) | idLivro` no `HashExtensivelLong`; `(idLivro << 16 | idAutor)` e `(idTag << 16 | idLivro)` nas Árvores B+ de N:N.
- **Chaves de unicidade:** `email` em `Usuarios` e `isbn` em `Livro`.

### e) Como as estruturas (hash) foram implementadas para cada chave de pesquisa?

Duas implementações distintas de Hash Extensível coexistem no projeto:

**HashExtensivel (chave int):** usada para busca direta por ID em todas as entidades. Entrada de 12 bytes ( `int` + `long` ).  
Função de hash: máscara dos bits menos significativos ( `chave & ((1 << profGlobal) - 1)` ).

**HashExtensivelLong (chave long):** usada exclusivamente no relacionamento 1:N. Entrada de 16 bytes ( `long` + `long` ).  
Implementa `listarPorFaixa(min, max)` que percorre todos os buckets coletando entradas no intervalo, habilitando o range scan por `idEditora` .

Ambas suportam divisão de bucket e duplicação de diretório de forma análoga.

### f) Como foi implementado o relacionamento 1:N (lógica de navegação e integridade referencial)?

O `LivroDAO` mantém `hashPorEditora` ( `HashExtensivelLong` ) com chave ( `idEditora << 32` ) | `idLivro` . Para listar livros de uma editora: `listarPorFaixa(idEditora << 32, (idEditora+1) << 32)` retorna os offsets de todos os livros vinculados, que são lidos diretamente do `livros.bin` . A **integridade referencial** é verificada no `EditoraDAO` antes de deletar. Ao alterar o `idEditora` de um livro, o `LivroDAO` remove a chave composta antiga e insere a nova, mantendo o índice sempre consistente.

### g) Como os índices são persistidos em disco? (formato, atualização, sincronização com os dados)

**Árvore B+ ( \*.idx ):** nós de tamanho fixo ( `TAM_NO` ), com cabeçalho de 32 bytes (endereço da raiz + total de chaves + reserva). Cada operação reescreve apenas os nós afetados via `RandomAccessFile.seek()` .

**Hash Extensível ( .dir.bin + .bkt.bin ):** o diretório armazena a profundidade global ( `int` ) seguida de ponteiros `long` para os buckets. Buckets têm tamanho fixo com contagem de entradas e array de pares ( `chave, endereço` ) . Toda operação de escrita nos dados (Create, Update, Delete) é imediatamente seguida pela atualização dos índices no mesmo fluxo de execução, sem transações separadas.

### h) Como está estruturado o projeto no GitHub?

```
TP-AEDS-III-G9/
├── src/main/java/src/
│   ├── dao/           # Um DAO por entidade – abstrai persistência e índices
│   ├── model/         # POJOs com toByteArray/fromByteArray (implementam Registro)
│   ├── server/        # ApiServer: roteamento HTTP e endpoints REST em Java puro
│   ├── service/       # Regras de negócio desacopladas (autenticação XOR, validações)
│   └── util/          # Estruturas: Arquivo<T>, ArvoreBMais, HashExtensivel,
│                       # HashExtensivelLong, OrdenacaoExterna, SerializadorUtil
├── web/               # Frontend SPA: index.html, app.js, style.css
├── data/              # Gerado em runtime: *.bin de dados + idx/ com índices
├── docs/              # Documentação das fases
└── pom.xml            # Maven – Java 17, org.json para API REST
```

Arquitetura **MVC**: `model/` (entidades), `dao/` + `util/` (persistência), `server/` (Controller REST), `web/` (View).

## 11. Como Compilar e Executar

**Pré-requisitos:** Java JDK 17+ e Maven 3.6+

```
# 1. Clonar o repositório
git clone https://github.com/puc-minas-aeds-iii/tp-g9.git
cd TP-AEDS-III-G9

# 2. Compilar (gera JAR fat com maven-shade-plugin)
mvn clean package

# 3. Executar o servidor
java -jar target/bibliosys-1.0-SNAPSHOT.jar

# 4. Acessar no navegador
# http://localhost:8080
```

## 12. Tecnologias Utilizadas



| Tecnologia                            | Finalidade                                                 |
|---------------------------------------|------------------------------------------------------------|
| Java 17                               | Backend, lógica de persistência e servidor HTTP nativo     |
| Maven + maven-shade-plugin            | Build e JAR executável com dependências embutidas          |
| <code>RandomAccessFile</code>         | Leitura/escrita direta e posicionada em arquivos binários  |
| <code>org.json</code>                 | Serialização/deserialização JSON na API REST               |
| HTML5 / CSS3 / JavaScript             | Interface web SPA (frontend vanilla)                       |
| Árvore B+                             | Indexação ordenada, travessia crescente/decrescente        |
| Hash Extensível ( <code>int</code> )  | Busca direta por ID — O(1) amortizado                      |
| Hash Extensível ( <code>long</code> ) | Relacionamento 1:N com range scan por editora              |
| Intercalação Balanceada               | Ordenação externa de grande volume com RAM fixa            |
| XOR + Base64                          | Criptografia manual de senhas com chave derivada do e-mail |

*Documentação produzida para a Fase 2 — entrega em 26/03/2026. Grupo 9 — AEDs III — PUC Minas.*